

Dynamic Programming: DP on Squares in a Matrix

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Dynamic Programming: DP on Squares in a Matrix	1
1.1	The Three-Neighbor Square DP	1
1.2	Count Square Submatrices with All Ones	2
1.3	Maximal Rectangle	3

Chapter 1

Dynamic Programming: DP on Squares in a Matrix

This closing chapter asks one last question about a binary matrix: how large a square (or rectangle) of all ones can be found, anchored at each cell? Once a cell knows the best answer for its neighbors above, left, and diagonally above-left, it can answer for itself in one comparison – no search over a range required.

1.1 The Three-Neighbor Square DP

The defining recurrence of this chapter looks at exactly three already-computed neighbors – directly above, directly left, and diagonally above-left – and takes their minimum: a square of all ones can only be extended by one row and one column simultaneously if *all three* of those smaller squares already existed.

The Two Problems in This Chapter

- **Count Square Submatrices with All Ones (Section 1.2):** the three-neighbor-minimum recurrence directly, summed over every cell.
- **Maximal Rectangle (Section 1.3):** a related but distinct DP – computing, for each cell, the height of consecutive ones *above* it (a simple 1D recurrence per column), which then feeds into the largest-rectangle-in-a-histogram technique already developed in the Stack and Queue chapter.

1.2 Count Square Submatrices with All Ones

Problem Statement. Given a binary matrix *mat*, count the total number of square submatrices that contain only 1s (squares of every size, at every position).

Pattern Recognition

“Largest/count of square submatrices of all ones, anchored at each cell” is solved by a recurrence that looks at exactly three neighbors: a square of side *s* ending at (i, j) as its bottom-right corner exists only if squares of side $s - 1$ already exist ending at **all three** of the cell above, the cell to the left, and the cell diagonally above-left.

Deriving the recurrence

Let $dp[i][j]$ be the side length of the **largest** all-ones square whose bottom-right corner is exactly (i, j) (and 0 if $mat[i][j] = 0$). If $mat[i][j] = 1$, extending a square one row down and one column right requires the square *ending* at each of the three neighboring corners to already support at least side length $dp[i][j] - 1$:

$$dp[i][j] = \begin{cases} 0 & mat[i][j] = 0, \\ 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) & mat[i][j] = 1. \end{cases}$$

Why the Minimum, and Why Summing Gives the Count

The new square’s side is capped by whichever of the three neighbors supports the *smallest* square – a single weak corner limits how far the square can extend, exactly like the weakest link in a chain. Separately, a cell with $dp[i][j] = k$ doesn’t just mean *one* square ends there – it means *k* different squares end there: one each of side $1, 2, \dots, k$ (every smaller square nested inside the largest one is also a valid, distinct all-ones square with its bottom-right corner at (i, j)). So the **count** of all square submatrices is simply $\sum_{i,j} dp[i][j]$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int countSquareSubmatrices(vector<vector<int>>& mat) {
5      int n = mat.size(), m = mat[0].size();
6      vector<vector<int>> dp(n, vector<int>(m, 0));
7      int totalCount = 0;
8
9      for (int i = 0; i < n; i++) {
10         for (int j = 0; j < m; j++) {
11             if (mat[i][j] == 0) {
12                 dp[i][j] = 0;
13             } else if (i == 0 || j == 0) {
14                 dp[i][j] = 1; // first row/column: at most a 1x1 square
15             } else {
16                 dp[i][j] = 1 + min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]});
17             }
18             totalCount += dp[i][j];
19         }
20     }

```

```

21     return totalCount;
22 }
23
24 int main() {
25     vector<vector<int>> mat = {
26         {0, 1, 1, 1},
27         {1, 1, 1, 1},
28         {0, 1, 1, 1}
29     };
30     cout << countSquareSubmatrices(mat) << endl; // 15
31     return 0;
32 }

```

Dry run

Dry Run. Take the 3×4 matrix above.

	0	1	2	3
0	0	1	1	1
1	1	1	2	2
2	2	0	1	2

$dp[1][2] = 2$: $mat[1][2] = 1$, and $\min(dp[0][2], dp[1][1], dp[0][1]) = \min(1, 1, 1) = 1$, so $dp[1][2] = 2$ (a 2×2 all-ones square exists with corners at rows 0–1, columns 1–2). $dp[2][3] = 3$: neighbors $dp[1][3] = 2$, $dp[2][2] = 2$, $dp[1][2] = 2$, minimum 2, giving $dp[2][3] = 3$ (a full 3×3 all-ones square, columns 1–3, all three rows). Summing every cell: $0+1+1+1+1+1+2+2+0+1+2+3 = 15$.

Complexity Analysis

Time Complexity. $O(n \cdot m)$ – one constant-time comparison per cell. **Space Complexity.** $O(n \cdot m)$ for the dp table (reducible to $O(m)$ with a rolling pair of rows, since each cell only reads the row directly above and the current row).

Key Takeaway

A surprisingly large class of “2D shape ending here” problems reduce to a three-neighbor minimum (or maximum) plus one: the moment a shape can only grow when *all* of its supporting smaller shapes already exist, the recurrence is this same one-line comparison, with no search over a range needed at all – the simplest recurrence shape in this entire book.

1.3 Maximal Rectangle

Problem Statement. Given a binary matrix mat , find the area of the largest rectangle (not necessarily square) containing only 1s.

Pattern Recognition

This closing problem of the entire book splits cleanly into two independent techniques: a simple per-column **height DP** (how many consecutive 1s sit directly above and including this cell), followed by treating each row’s height array as a **histogram** and finding its largest rectangle – exactly the monotonic-stack technique already developed for Largest Rectangle in Histogram in the Stack and Queue chapter.

Deriving the height DP

Let $height[i][j]$ be the number of consecutive 1s ending at row i , looking upward, in column j :

$$height[i][j] = \begin{cases} 0 & mat[i][j] = 0, \\ height[i-1][j] + 1 & mat[i][j] = 1. \end{cases}$$

This is the same “reset to 0 on failure, otherwise extend by 1” shape seen throughout this book (Longest Common Substring’s mismatch reset, House Robber’s running totals) – here applied independently to every column.

Reframing each row as a histogram

After computing $height[i][\cdot]$ for a fixed row i , that row *is* a histogram: $height[i][j]$ is the bar height at position j . The largest all-ones rectangle with its **bottom edge on row i** is exactly the largest rectangle in that histogram – solved in $O(m)$ using the monotonic-stack technique from the Stack and Queue chapter’s Largest Rectangle in Histogram problem. Running this once per row and tracking the global maximum solves the full problem.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int largestRectangleInHistogram(vector<int>& heights) {
5      int n = heights.size();
6      stack<int> st;
7      int maxArea = 0;
8
9      for (int i = 0; i <= n; i++) {
10         int currentHeight = (i == n) ? 0 : heights[i];
11         while (!st.empty() && heights[st.top()] >= currentHeight) {
12             int height = heights[st.top()];
13             st.pop();
14             int width = st.empty() ? i : i - st.top() - 1;
15             maxArea = max(maxArea, height * width);
16         }
17         st.push(i);
18     }
19     return maxArea;
20 }
21
22 int maximalRectangle(vector<vector<int>>& mat) {
23     int n = mat.size(), m = mat[0].size();
24     vector<int> height(m, 0);
25     int best = 0;
26
27     for (int i = 0; i < n; i++) {
28         for (int j = 0; j < m; j++) {
29             height[j] = (mat[i][j] == 1) ? height[j] + 1 : 0;
30         }
31         best = max(best, largestRectangleInHistogram(height));
32     }
33     return best;
34 }
35

```

```

36 int main() {
37     vector<vector<int>> mat = {
38         {1, 0, 1, 0, 0},
39         {1, 0, 1, 1, 1},
40         {1, 1, 1, 1, 1},
41         {1, 0, 0, 1, 0}
42     };
43     cout << maximalRectangle(mat) << endl; // 6
44     return 0;
45 }

```

Dry run

Dry Run. Take the 4×5 matrix above.

Row-by-row height arrays: row 0: [1, 0, 1, 0, 0]. row 1: [2, 0, 2, 1, 1]. row 2: [3, 1, 3, 2, 2]. row 3: [4, 0, 0, 3, 0].

At row 2, the histogram [3, 1, 3, 2, 2] has its largest rectangle as columns 2–4 at height 2 (area 6): bars of height 3, 2, 2, the minimum height 2 spanning width 3, giving area 6. This is the overall maximum: at row 0 the best is area 1; at row 1, best is $3 \times 1 = 3$ (columns 2–4, heights 2, 1, 1, limited by the minimum height 1); at row 3, best is 4 (column 0 alone, height 4). The global maximum across all rows is 6.

Complexity Analysis

Time Complexity. $O(n \cdot m)$ – n rows, each requiring an $O(m)$ height update and an $O(m)$ histogram pass (the monotonic stack visits each bar a bounded number of times). **Space Complexity.** $O(m)$ for the *height* array and the stack.

Key Takeaway

This book closes on a problem that is itself a closing argument for everything in it: recognizing that a 2D problem decomposes into “a simple DP recurrence run independently per column” *composed with* “a classical 1D technique from an earlier chapter, run once per row” is, in the end, the same skill exercised throughout this entire book – noticing which already-solved shape a new problem’s structure matches, rather than deriving a solution from nothing.